

BPA-KOM Lab 4 -TCP and UDP

Name	Dave Galea
VUT ID	284844
Lab Number	4
Date	December 2025

1. Objective 1

Task Assignment: The aim of this task was to analyse UDP packets using the DNS protocol in Wireshark.

Solution:

After starting to capture packets in Wireshark, in CMD, the `nslookup` command was executed which takes the executer to the interactive mode. `type=A` was set to query only IPv4 addresses, and the IP address for the domain was set as `www.vut.cz`. The result of the above is the following:

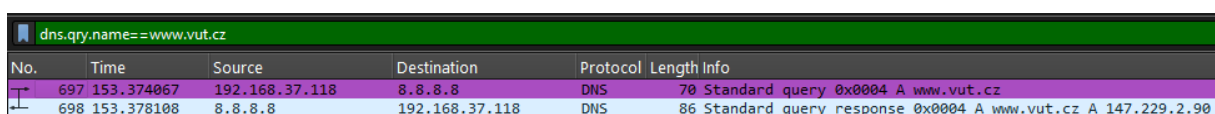
```
C:\Users\Student>nslookup
Default Server:  dns.google
Address:  8.8.8.8

> set type=A
> www.vut.cz
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    www.vut.cz
Address:  147.229.2.90
```

Figure 1: Output of `nslookup`.

In Wireshark, the Filter was set to the following: `dns.qry.name==www.vut.cz.localdomain`. This displays only the DNS query-response that was issued. The output after setting the filter is the following:



dns.qry.name==www.vut.cz						
No.	Time	Source	Destination	Protocol	Length	Info
697	153.374067	192.168.37.118	8.8.8.8	DNS	70	Standard query 0x0004 A www.vut.cz
698	153.378108	8.8.8.8	192.168.37.118	DNS	86	Standard query response 0x0004 A www.vut.cz A 147.229.2.90

Figure 2: Output in Wireshark after setting the stated filter.

Upon expanding the query message, it can be noted that UDP is used as the transport layer protocol. After expanding User Datagram Protocol section, the output is the following:

```
▼ User Datagram Protocol, Src Port: 57430, Dst Port: 53
  Source Port: 57430
  Destination Port: 53
  Length: 36
  Checksum: 0xf663 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 61]
  [Stream Packet Number: 1]
  ▶ [Timestamps]
  UDP payload (28 bytes)
```

Figure 3: Expansion of User Datagram Protocol section of query message

The output matches what is discussed in the introduction; the length is 36 bytes, while the payload (data) is 28bytes, meaning that the header is 8 bytes. This is because the header consists of 4 sections, each of 2 bytes each; the Source Port, the Destination port, the Length, and the Checksum.

Upon expanding the response message, the output is as following:

```
▼ User Datagram Protocol, Src Port: 53, Dst Port: 57430
  Source Port: 53
  Destination Port: 57430
  Length: 52
  Checksum: 0xe01a [unverified]
  [Checksum Status: Unverified]
  [Stream index: 61]
  [Stream Packet Number: 2]
  ▶ [Timestamps]
  UDP payload (44 bytes)
```

Figure 4: Expansion of User Datagram Protocol section of response message

When comparing the expansion of the respective User Datagram Protocol sections of the query and response messages, it can be noted that the source and destination ports swap values. In both, the UDP header size still remains the same value of 8 bytes.

2. Objective 2

Task Assignment: The aim of this task was to analyse TCP packets using the DNS protocol in Wireshark.

Solution:

After typing `set vc` in CMD so to enforce DNS to establish virtual circuit and use TCP, the query for `www.vut.cz` was issued again. With the same filter as before still applied, it can be seen in the output below that 2 new records appear.

No.	Time	Source	Destination	Protocol	Length	Info
697	153.374067	192.168.37.118	8.8.8.8	DNS	70	Standard query 0x0004 A www.vut.cz
698	153.378108	8.8.8.8	192.168.37.118	DNS	86	Standard query response 0x0004 A www.vut.cz A 147.229.2.90
9582	1944.247695	192.168.37.118	8.8.8.8	DNS	82	Standard query 0x0009 A www.vut.cz
9584	1944.251611	8.8.8.8	192.168.37.118	DNS	100	Standard query response 0x0009 A www.vut.cz A 147.229.2.90

Figure 5: Output in Wireshark after typing `set vc` in CMD.

Upon right clicking on one of the new messages, followed by **Follow > TCP Stream**, one can observe the captured TCP steam.

No.	Time	Source	Destination	Protocol	Length	Info
9575	1944.238935	192.168.37.118	8.8.8.8	TCP	66	63691 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
9578	1944.243020	8.8.8.8	192.168.37.118	TCP	66	53 → 63691 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
9579	1944.243079	192.168.37.118	8.8.8.8	TCP	54	63691 → 53 [ACK] Seq=1 Ack=1 Win=65280 Len=0
9580	1944.243707	192.168.37.118	8.8.8.8	TCP	56	63691 → 53 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=2 [TCP PDU reassembled in 9582]
9581	1944.247673	8.8.8.8	192.168.37.118	TCP	60	53 → 63691 [ACK] Seq=1 Ack=3 Win=269824 Len=0
9582	1944.247695	192.168.37.118	8.8.8.8	DNS	82	Standard query 0x0009 A www.vut.cz
9583	1944.251582	8.8.8.8	192.168.37.118	TCP	60	53 → 63691 [ACK] Seq=1 Ack=31 Win=269824 Len=0
9584	1944.251611	8.8.8.8	192.168.37.118	DNS	100	Standard query response 0x0009 A www.vut.cz A 147.229.2.90
9585	1944.251826	192.168.37.118	8.8.8.8	TCP	54	63691 → 53 [FIN, ACK] Seq=31 Ack=47 Win=65280 Len=0
9586	1944.255999	8.8.8.8	192.168.37.118	TCP	60	53 → 63691 [FIN, ACK] Seq=47 Ack=32 Win=269824 Len=0
9587	1944.256049	192.168.37.118	8.8.8.8	TCP	54	63691 → 53 [ACK] Seq=32 Ack=48 Win=65280 Len=0

Figure 6: Captured TCP steam using DNS.

The first 3 messages in the TCP stream were exchanged so to establish the virtual circuit. Upon examining the first message, the output is as follows:

Transmission Control Protocol, Src Port: 63691, Dst Port: 53, Seq: 0, Len: 0 Source Port: 63691 Destination Port: 53 [Stream index: 131] [Stream Packet Number: 1] [Conversation completeness: Complete, WITH_DATA (31)] [TCP Segment Len: 0] Sequence Number: 0 (relative sequence number) Sequence Number (raw): 115590891 [Next Sequence Number: 1 (relative sequence number)] Acknowledgment Number: 0 Acknowledgment number (raw): 0 1000 = Header Length: 32 bytes (8) [Flags: 0x002 (SYN)] Window: 65535 [Calculated window size: 65535] Checksum: 0xf654 [unverified] [Checksum Status: Unverified] Urgent Pointer: 0 [Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted] [Timestamps]
--

Figure 7: First TCP message

The non optional parts of the TCP header include: Source port, Destination port, Sequence number, Acknowledgement number, Header length, and reserved. These together occupy 20 bytes. In the above output, it can be seen that the Options are 12 bytes, and the total header length is 32 bytes, meaning that the rest of the header has a corresponding length of 20 bytes.

The remaining two messages are as follows:

No.	Time	Source	Destination	Protocol	Length	Info
9575	1944.238935	192.168.37.118	8.8.8.8	TCP	66	63691 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK PERM
9578	1944.243020	8.8.8.8	192.168.37.118	TCP	66	53 → 63691 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK PERM WS=256
9579	1944.243079	192.168.37.118	8.8.8.8	TCP	54	63691 → 53 [ACK] Seq=1 Ack=1 Win=65280 Len=0
9580	1944.243707	192.168.37.118	8.8.8.8	TCP	56	63691 → 53 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=2 [TCP PDU reassembled in 95
9581	1944.247673	8.8.8.8	192.168.37.118	TCP	60	53 → 63691 [ACK] Seq=1 Ack=3 Win=269824 Len=0

Figure 8: Remaining two messages.

It can be seen that they have the appropriate description of a three-way handshake with the 1st message as follows:

1. Client - [SYN]
2. Server - [SYN, ACK]
3. Client - [ACK]

The TCP section inside the packet can be observed as follows:

▼ Transmission Control Protocol, Src Port: 63691, Dst Port: 53, Seq: 3, Ack: 1, Len: 28
Source Port: 63691
Destination Port: 53
[Stream index: 131]
[Stream Packet Number: 6]
▶ [Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 28]
Sequence Number: 3 (relative sequence number)
Sequence Number (raw): 115590894
[Next Sequence Number: 31 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 3882301705
0101 = Header Length: 20 bytes (5)
▶ Flags: 0x018 (PSH, ACK)
Window: 255
[Calculated window size: 65280]
[Window size scaling factor: 256]
Checksum: 0xf664 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
▶ [Timestamps]
▶ [SEQ/ACK analysis]
TCP payload (28 bytes)
[PDU Size: 30]
TCP segment data (28 bytes)

Figure 9: TCP section inside the packet

And the TCP section inside the previous neighbouring packet can be observed as follows:

```
▼ Transmission Control Protocol, Src Port: 53, Dst Port: 63691, Seq: 1, Ack: 3, Len: 0
  Source Port: 53
  Destination Port: 63691
  [Stream index: 131]
  [Stream Packet Number: 5]
  ▶ [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 1      (relative sequence number)
  Sequence Number (raw): 3882301705
  [Next Sequence Number: 1      (relative sequence number)]
  Acknowledgment Number: 3      (relative ack number)
  Acknowledgment number (raw): 115590894
  0101 .... = Header Length: 20 bytes (5)
  ▶ Flags: 0x010 (ACK)
  Window: 1054
  [Calculated window size: 269824]
  [Window size scaling factor: 256]
  Checksum: 0xce44 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]
```

Figure 10: TCP section inside the previous neighbouring packet

Comparing the Sequence number and Acknowledgement number of the two, it can be seen that sequence number of one packet is the acknowledgement number of the other, and vice versa. This is because TCP acknowledges the last byte received and indicates the next byte expected, enabling reliable two-way communication.

The DNS server then acknowledges the incoming query, and sets the acknowledgement number to 31, meaning that it has successfully received the first 30 bytes from the client and expects the next byte starting at 31.

```
▼ Transmission Control Protocol, Src Port: 53, Dst Port: 63691, Seq: 1, Ack: 31, Len: 46
  Source Port: 53
  Destination Port: 63691
  [Stream index: 131]
  [Stream Packet Number: 8]
  ▶ [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 46]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 3882301705
  [Next Sequence Number: 47 (relative sequence number)]
  Acknowledgment Number: 31 (relative ack number)
  Acknowledgment number (raw): 115590922
  0101 .... = Header Length: 20 bytes (5)
  ▶ Flags: 0x018 (PSH, ACK)
  Window: 1054
  [Calculated window size: 269824]
  [Window size scaling factor: 256]
  Checksum: 0x8559 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]
  TCP payload (46 bytes)
  [PDU Size: 46]
```

Figure 11: DNS server standard query response

The following 4 packets after the response represent the four-way handshake to close the connection; in the output of Wireshark, only three packets came up, however the flags set still resemble the process in the **abbreviated** form. The flags set were:

1. Server - [FIN, ACK]
2. Client - [FIN, ACK]
3. Server - [ACK]

This is a faster alternative to the normal four-way handshake, which would have an additional Client - [ACK] after receiving from the server.

In CMD, the `nslookup` behaviour was set to the default state of using UDP for simple transmission by typing `set novc`.

3. Objective 3

Task Assignment: The aim of this task was to compare the two protocols by means of graphs.

Solution:

After using the same `dns.qry.name==www.vut.cz.localdomain` filter to display DNS

conversations, one of the packets that used UDP as the transport protocol was selected. It was right clicked and **Follow > UDP Stream** was selected. The specified packets were exported.

The above process was then repeated for the TCP stream, and then the two were merged.

Following are the graphs comparing Packets for UDP and TCP

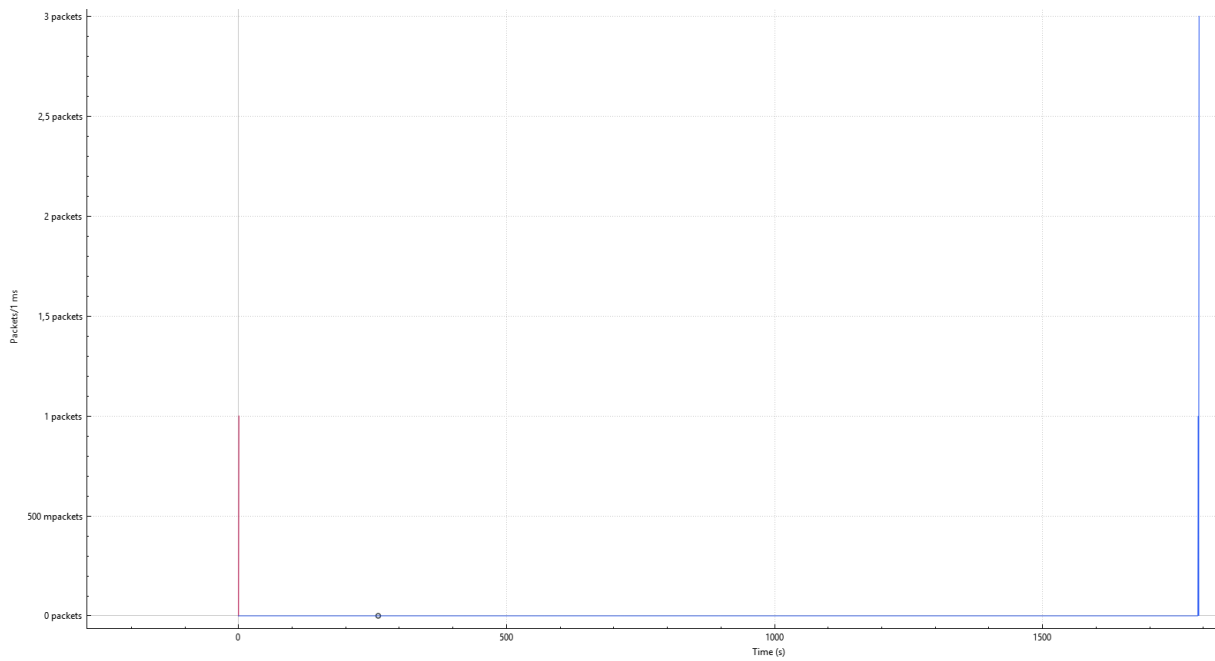


Figure 12: Packets sent during DNS transmissions using UDP(red) and TCP(blue)

In the Lab we were informed that there was a bug with Wireshark during the Lab, however, from the output, it can still be shown that more packets were used for the complete transmission in case of TCP (blue) than with UDP. The largest amount of packets per second for TCP was 3, and 1 for UDP.

Following are the graphs comparing Bytes for UDP and TCP:

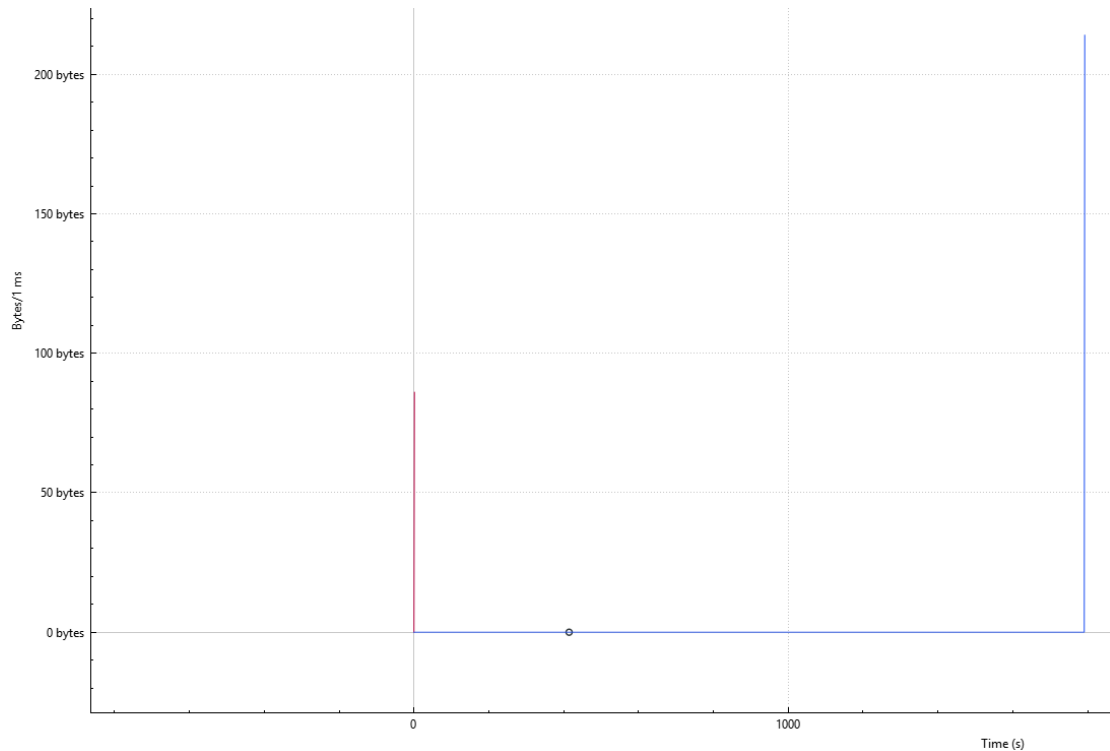


Figure 13: Bytes sent during DNS transmission using UDP (red) and TCP (blue)

Once again, the output was bugged, however, it can still be seen that TCP transferred many more bytes per second; reaching 210 at the peak, while UDP reached 80.

Although Throughput of TCP was higher, in terms of time, UDP was faster in full query-response transfer.

4. Objective 4

Task Assignment: The aim of this task was to generate HTTP traffic in Wireshark.

Solution:

To do this, the website <http://www.neverssl.com> was visited, and after the content was loaded, the browser was closed and capturing in Wireshark was stopped.

After, any of the packets was right clicked and the TCP Stream was observed. A new pop up window with HTML structure opens as follows:


```

SET /fog/management/index.php?sub=requestClientInfo&configure&newService&json HTTP/1.1
Host: fog.utko.feec.vutbr.cz
Connection: Keep-Alive

HTTP/1.1 200 OK
Date: Wed, 03 Dec 2025 11:35:56 GMT
Server: Apache/2.4.58 (Ubuntu)
X-Frame-Options: sameorigin
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000
Content-Security-Policy: default-src 'none';script-src 'self' 'unsafe-eval';connect-src 'self';img-src 'self' data:;style-src 'self' 'unsafe-inline';font-src 'self';
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=smq1pkhucidn29ps05r338eu3r; path=/; HttpOnly; SameSite=Lax
Vary: Accept-Encoding
Connection: close
Transfer-Encoding: chunked
Content-Type: text/html; charset=UTF-8

88
{"sleep":121,"maxsize":"20480000","promptTime":"60","force":false,"bannerURL":"","bannerHash":"","color":"","company":"VUT FEKT UTKO"}

```

Figure 14: Window with HTML structure

The TCP packets were then observed. There was some data loss during transmission, as indicated by the black highlighted rows in the output below.

438	5.212147	192.168.37.118	34.223.124.45	TCP	66 61477 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
440	5.393689	34.223.124.45	192.168.37.118	TCP	66 443 → 61477 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1460 SACK_PERM WS=4096
442	5.397606	192.168.37.118	34.223.124.45	TCP	66 61478 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
444	5.578980	34.223.124.45	192.168.37.118	TCP	66 443 → 61478 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1460 SACK_PERM WS=4096
477	6.202854	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 63035 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
478	6.202864	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 53789 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
501	8.203479	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 53789 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
502	8.204237	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 63035 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
525	12.204560	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 53789 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
526	12.220084	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 63035 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
531	12.789503	192.168.37.118	142.251.38.138	TCP	66 58906 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
532	12.793681	142.251.38.138	192.168.37.118	TCP	66 443 → 58906 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
590	20.209902	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 53789 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
591	20.229326	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 63035 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
624	27.282495	192.168.37.118	34.223.124.45	TCP	66 59958 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
625	27.283962	192.168.37.118	34.223.124.45	TCP	66 51267 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
639	28.296208	192.168.37.118	34.223.124.45	TCP	66 [TCP Retransmission] 59958 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
655	30.096841	192.168.37.118	142.251.38.131	TCP	66 63418 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
658	30.100938	142.251.38.131	192.168.37.118	TCP	66 443 → 63418 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
666	30.118996	192.168.37.118	142.250.147.188	TCP	66 56452 → 5228 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
676	30.136989	192.168.37.118	142.251.38.131	TCP	66 55140 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
699	30.139635	192.168.37.118	142.251.38.131	TCP	66 63884 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
701	30.141024	142.251.38.131	192.168.37.118	TCP	66 443 → 55140 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
727	30.143666	142.251.38.131	192.168.37.118	TCP	66 443 → 63884 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
730	30.145040	142.250.147.188	192.168.37.118	TCP	66 5228 → 56452 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
764	30.184985	192.168.37.118	142.251.38.131	TCP	66 [TCP Dup ACK 762#1] 63884 → 443 [ACK] Seq=343 Ack=3191 Win=65280 Len=0 SLE=2825 SRE=3191
765	30.185619	192.168.37.118	142.251.36.78	TCP	66 53048 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
770	30.189715	142.251.36.78	192.168.37.118	TCP	66 443 → 53048 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256

Figure 15: Packet loss during the TCP transmission.

In my output, I did not have any message **TCP Previous segment not captured**. In the Lab it is asked why sequence number 1 is still expected from the client after such a message; this is because TCP only acknowledges data that arrives in order, so even though a higher sequence number packet arrived, the client continues to request sequence number 1.

In my output, i had messages **TCP Dup ACK** and **TCP Retransmission**. **TCP Dup ACK** means that the receiver sent the same acknowledgment number again, indicating that it is still expecting the same sequence number because some data has not been received. **TCP Retransmission** means that the sender retransmitted a data segment because it detected packet loss, usually after receiving duplicate acknowledgments (TCP Dup ACK).

5. Objective 5

Task Assignment: The aim of this task was to create and configure the reference topology in Cisco Packet Tracer along with adding the new network to OSPF routing process.

Solution:

After adding the Server-PT device and adding it to the topology saved from previous labs, the topology is as follows:

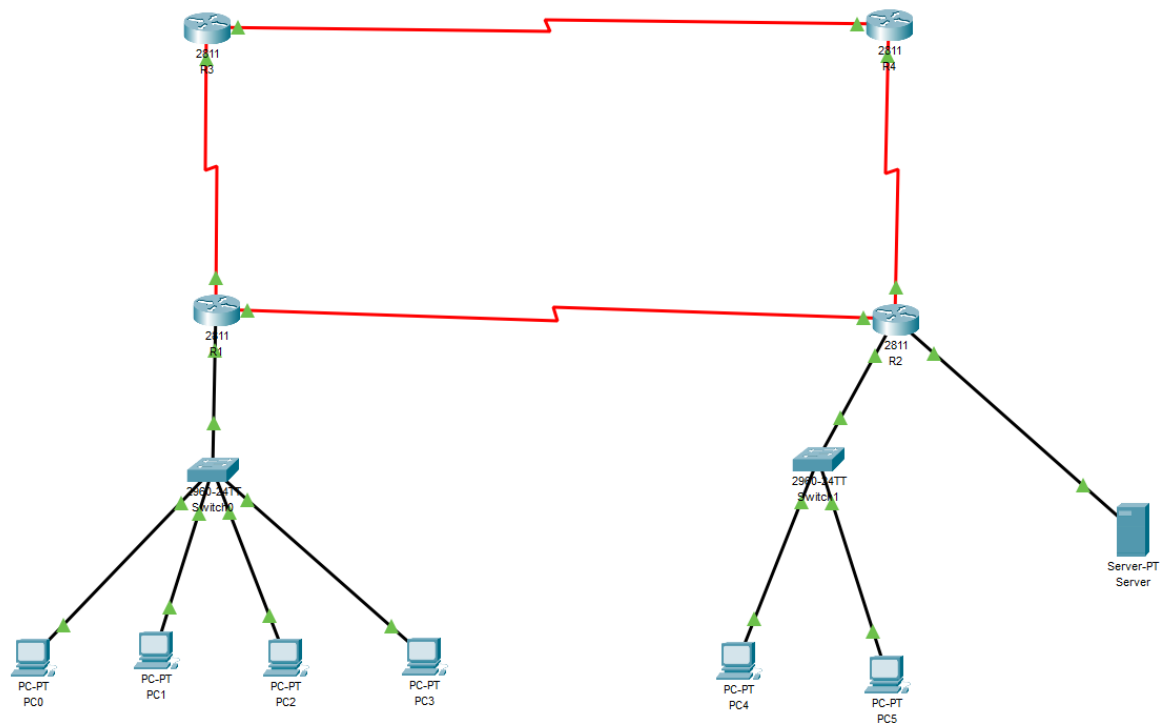


Figure 16: Current Topology in Cisco Packet Tracer.

After configuring the server and the router interface, R1's routing table was explored. The last entry of the table can be seen to be configured via RIP; this is because of the configuration from the last Lab.

```

R1>enable
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route
|
Gateway of last resort is not set

      172.20.0.0/16 is variably subnetted, 10 subnets, 3 masks
C       172.20.0.0/24 is directly connected, FastEthernet0/0
L       172.20.0.254/32 is directly connected, FastEthernet0/0
O       172.20.1.0/24 [110/193] via 172.20.2.6, 00:24:45, Serial0/3/1
C       172.20.2.0/30 is directly connected, Serial0/3/0
L       172.20.2.1/32 is directly connected, Serial0/3/0
C       172.20.2.4/30 is directly connected, Serial0/3/1
L       172.20.2.5/32 is directly connected, Serial0/3/1
O       172.20.2.8/30 [110/128] via 172.20.2.6, 00:24:45, Serial0/3/1
O       172.20.2.12/30 [110/192] via 172.20.2.6, 00:24:45, Serial0/3/1
R       172.20.3.0/24 [120/1] via 172.20.2.2, 00:00:23, Serial0/3/0

```

Figure 17: R1 routing table with RIP path.

So to have routes learnt only via OSPF in the routing tables, *172.20.3.0/24* subnet was added to the OSPF routing process. After this, R1's routing table was displayed again, and it can be seen that the RIP entry is now replaced by the OSPF entry.

```

R1#
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

      172.20.0.0/16 is variably subnetted, 10 subnets, 3 masks
C       172.20.0.0/24 is directly connected, FastEthernet0/0
L       172.20.0.254/32 is directly connected, FastEthernet0/0
O       172.20.1.0/24 [110/193] via 172.20.2.6, 00:35:12, Serial0/3/1
C       172.20.2.0/30 is directly connected, Serial0/3/0
L       172.20.2.1/32 is directly connected, Serial0/3/0
C       172.20.2.4/30 is directly connected, Serial0/3/1
L       172.20.2.5/32 is directly connected, Serial0/3/1
O       172.20.2.8/30 [110/128] via 172.20.2.6, 00:35:12, Serial0/3/1
O       172.20.2.12/30 [110/192] via 172.20.2.6, 00:35:12, Serial0/3/1
O       172.20.3.0/24 [110/193] via 172.20.2.6, 00:00:07, Serial0/3/1

```

Figure 18: R1 routing table with OSPF path.

The reachability of the server to R1's LAN was tested via the `ping` command. It was successful.

```
C:\>ping 172.20.0.254

Pinging 172.20.0.254 with 32 bytes of data:

Reply from 172.20.0.254: bytes=32 time=77ms TTL=252
Reply from 172.20.0.254: bytes=32 time=44ms TTL=252
Reply from 172.20.0.254: bytes=32 time=3ms TTL=252
Reply from 172.20.0.254: bytes=32 time=3ms TTL=252

Ping statistics for 172.20.0.254:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 3ms, Maximum = 77ms, Average = 31ms
```

Figure 19: ping command to test reachability of server to R1's LAN.

6. Objective 6

Task Assignment: The aim of this task was to start the DNS and HTTP services and create a web page on the server.

Solution:

By altering the configuration of services of the server, it was made as the DNS server and the web server for R1's LAN.

First, the HTTP was set up after editing the `index.html` file appropriately:

HTTP

HTTP
☒ On ☐ Off

HTTPS
☐ On ☒ Off

File Manager

	File Name	Edit	Delete
1	index.html	(edit)	(delete)

Figure 20: HTTP service setup.

Then, the DNS service was configured as follows:

The screenshot shows a DNS configuration window. At the top, 'DNS Service' is set to 'On'. Below it, the 'Resource Records' section has input fields for 'Name' and 'Type' (set to 'A Record'), and an 'Address' field. There are 'Add', 'Save', and 'Remove' buttons. A table below shows one record:

No.	Name	Type	Detail
0	www.mywebpage.com	A Record	172.20.3.1

Figure 21: DNS server configuration.

7. Objective 7

Task Assignment: The aim of this task was to configure the DNS server address on the PCs.

Solution:

So that the clients know which server to use for domain name translations, the DNS server address was set on the PCs of R1's LAN. The following is an example of one of the PCs.

The screenshot shows the 'IP Configuration' window. The 'Static' radio button is selected. The settings are as follows:

Field	Value
IPv4 Address	172.20.0.4
Subnet Mask	255.255.255.0
Default Gateway	172.20.0.254
DNS Server	172.20.3.1

Figure 22: DNS Server set up on PC of R1's LAN.

8. Objective 8

Task Assignment: The aim of this task was to access the server web page and analyse the traffic.

Solution:

For this objective, **Simulation** mode was activated, and the following protocols were selected for capturing:

- UDP
- TCP
- DNS
- HTTP

From a Pc from R1's LAN, `www.mywebpage.com` was entered into the Web Bowser. The webpage prints as follows:

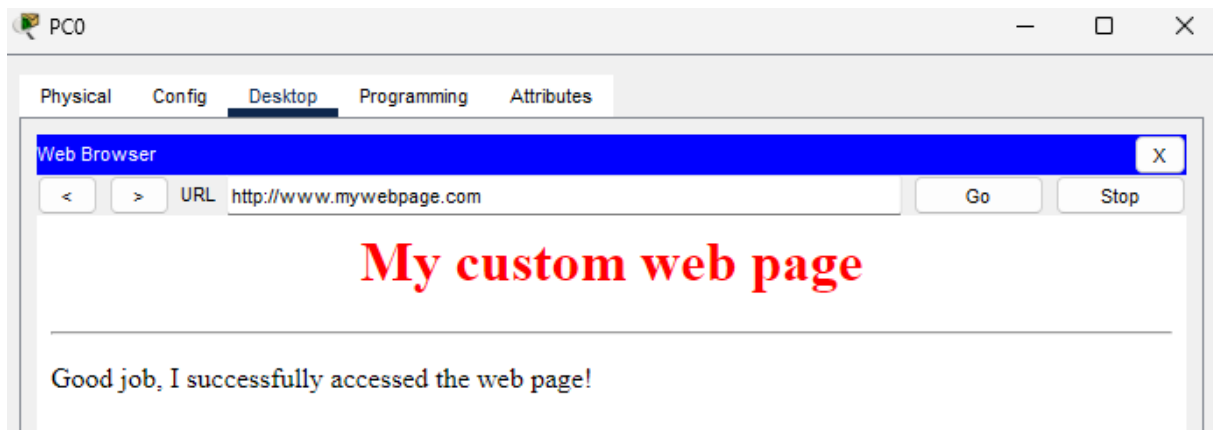


Figure 23: Content of the webpage after formatting index.html.

This prompts a DNS datagram to appear in the event list. This is the content:

Out Layers

Layer 7: DNS
Layer 6
Layer 5
Layer 4: UDP Src Port: 1028, Dst Port: 53
Layer 3: IP Header Src. IP: 172.20.0.1, Dest. IP: 172.20.3.1
Layer 2: Ethernet II Header 0060.3ECE.6DDC >> 0002.1645.B401
Layer 1: Port(s): FastEthernet0

Figure 24: Content of the DNS datagram.

It can be seen that the source port is 1028, and the Destination port is 52. The destination IP address is 172.20.3.1

Capture then forward was then clicked until the datagram reaches the server. When it reaches, this is the outbound section:

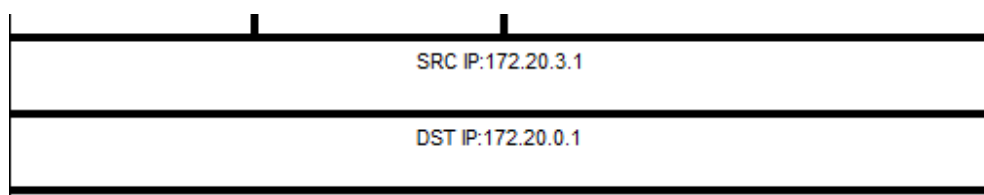


Figure 25: Outbound section of the datagram reaching the server.

It can be seen that the Source and Destination ports, as happened in Wireshark, swap places with respect to the previous datagram.

The DNS answer yields the following:

DNS Answer															
0															
															Bits
NAME (VARIABLE LENGTH):www.mywebpage.com															
TYPE:1								CLASS:1							
TTL:86400															
LENGTH:4								IP:172.20.3.1							

Figure 26: DNS answer output.

The IP address the server is responding for the queried domain is 172.20.3.1.

Capture then forward is then clicked again until the datagram reaches the PC. A new TCP segment is generated and the content is the following:

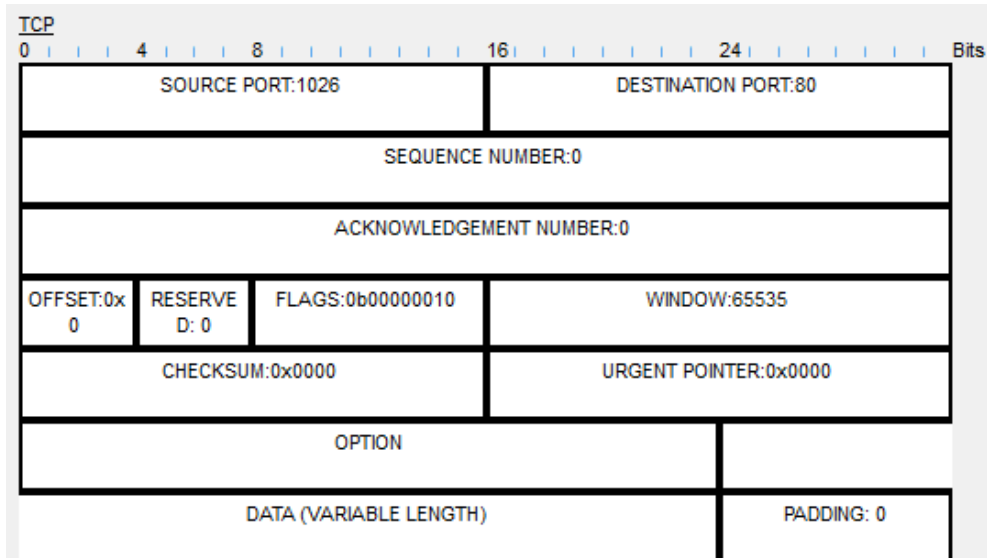


Figure 27: Content of newly generated TCP segment.

From the content, the following can be seen:

- Source port: 1026
- Destination port: 80
- Sequence number: 0
- Acknowledgment number: 0
- Flags set: 1 (SYN)

Capture then forward is then clicked again until the datagram reaches the server. The content is the following:

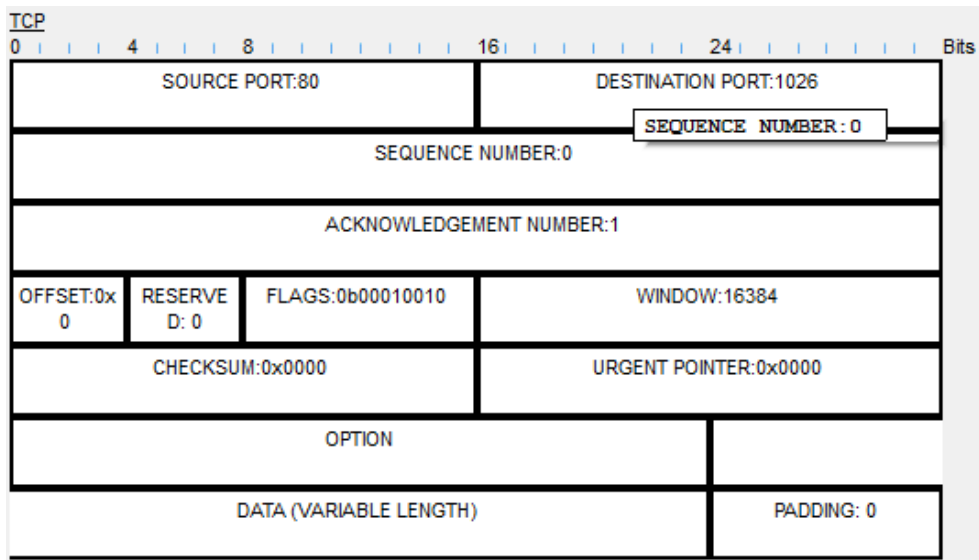


Figure 28: Content of the datagram reaching the server.

From the content, the following can be seen:

- Source port: 80
- Destination port: 1026
- Sequence number: 0
- Acknowledgment number: 1
- Flags set: 2 (SYN and ACK)

Capture then forward is then clicked again until the datagram reaches the PC once again. The content is the following:

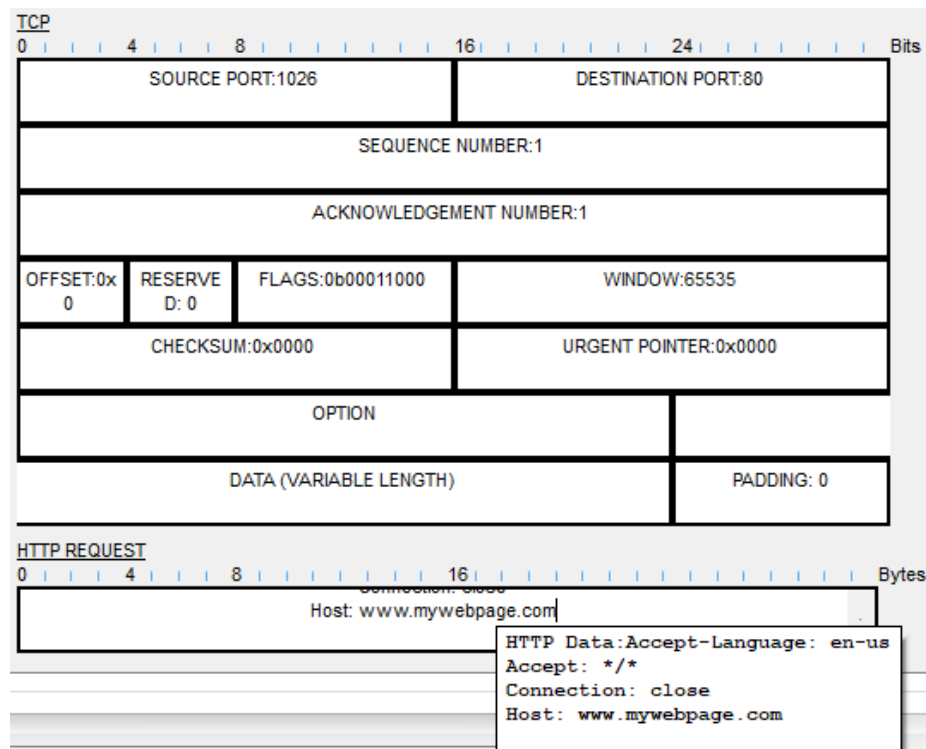


Figure 29: Content of the datagram reaching the PC once again.

From the content, the following can be seen:

- Source port: 1026
- Destination port: 80
- Sequence number: 1
- Acknowledgment number: 1
- Flags set: 2 (ACK and PSH)
- HTTP message transferred: REQUEST

This part represents the end of the three-way handshake, and HTTP messages are now transferred.

Capture then forward is then clicked again until the datagram reaches the server. Both the TCP segment finishing the three-way handshake and HTTP message are now transferred

Once more, Capture then forward is clicked so that the HTTP message reaches the server. The contents are the following

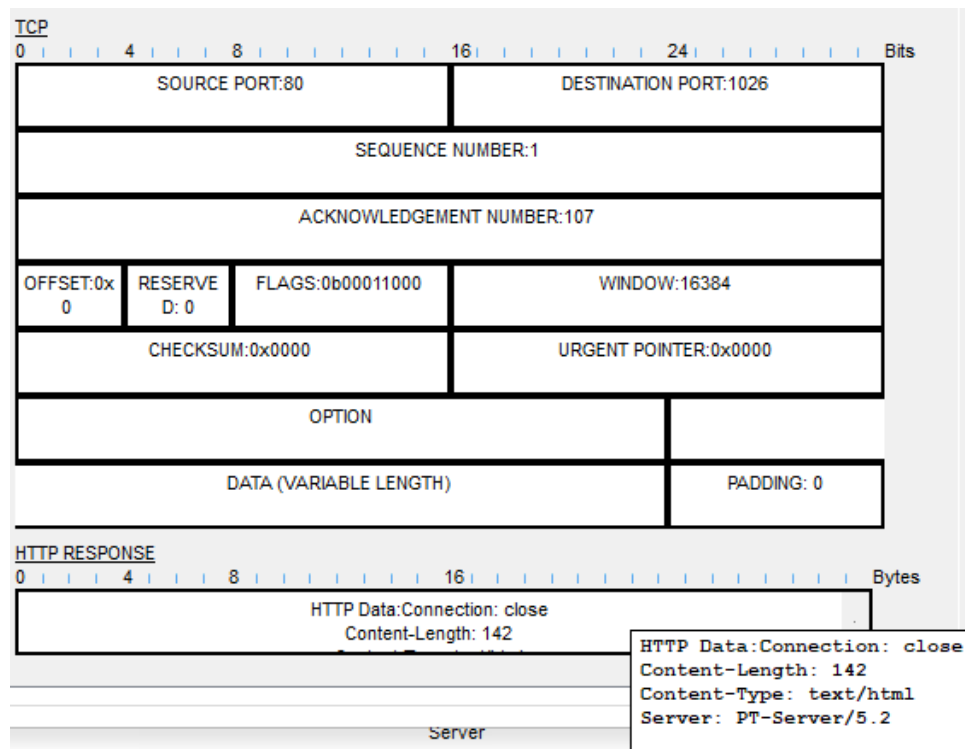


Figure 30: Content of Outbound PDU details.

From the content, the following can be seen:

- Source port: 80
- Destination port: 1026
- Sequence number: 1
- Acknowledgment number: 107
- Flags set: 2 (ACK and PSH)
- HTTP message transferred: RESPONSE

Capture then forward until HTTP message reaches the PC is performed, and a new TCP segment is generated.

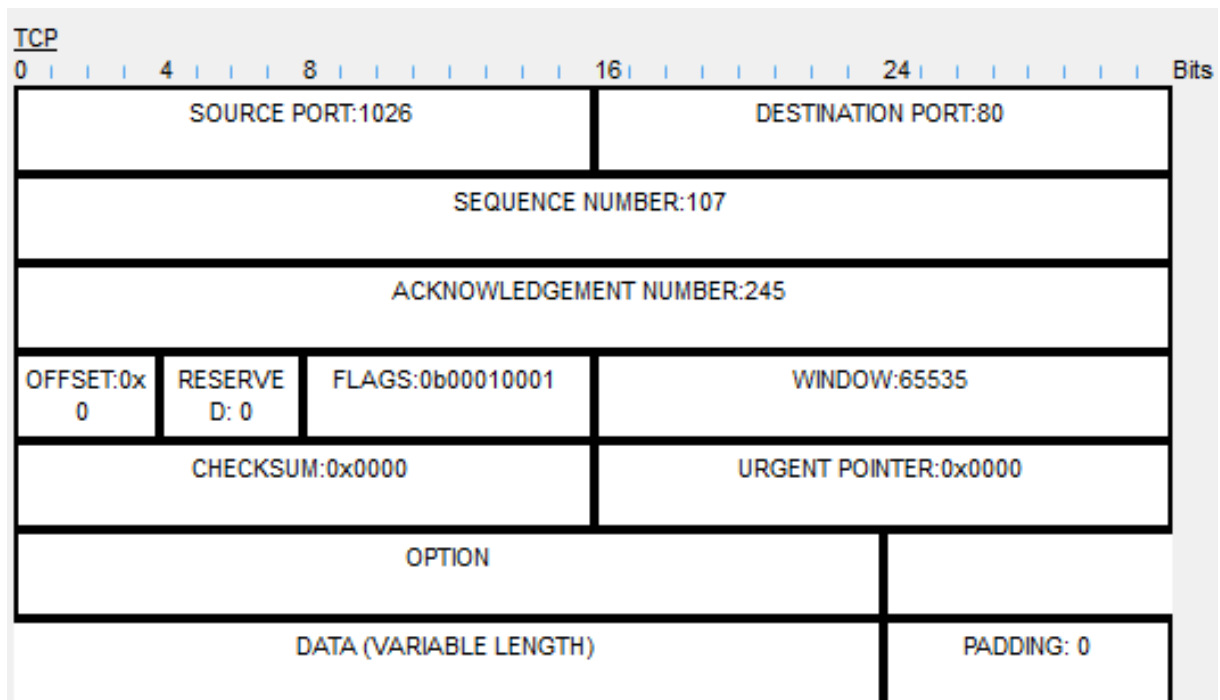


Figure 31: Content of Outbound PDU details at the client.

From the content, the following can be seen:

- Source port: 1026
- Destination port: 80
- Sequence number: 107 (previously 1)
- Acknowledgment number: 245 (previously 107)
- Flags set: 2 (FIN and ACK)
- HTTP message transferred: None as web page was successfully transferred.

Capture then forward is then performed until the segment reaches the server, and contents are the following:

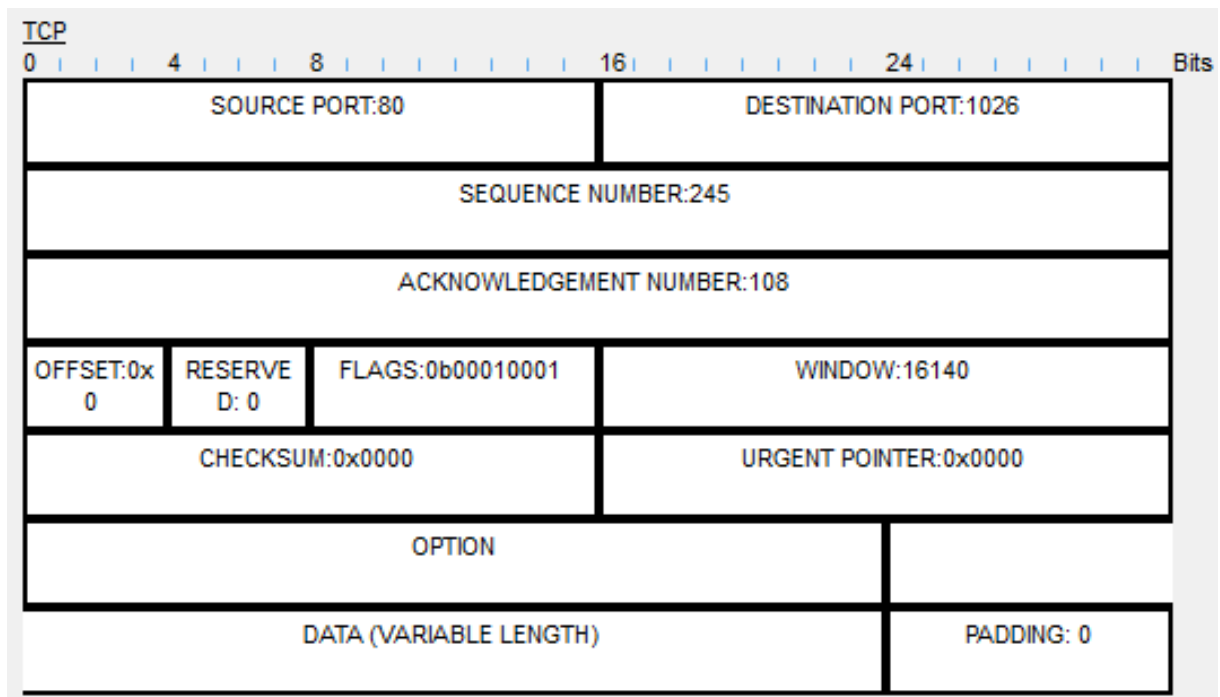


Figure 32: Content of Outbound PDU details at the server.

From the content, the following can be seen:

- Source port: 80
- Destination port: 1026
- Sequence number: 245
- Acknowledgment number: 108
- Flags set: 2 (FIN and ACK)

For the final time, capture then forward is performed until the segment reaches the client. The contents of the segment are the following:

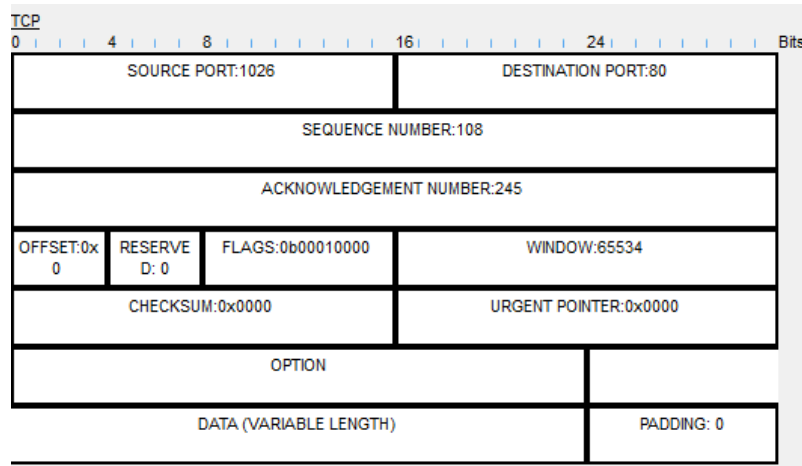


Figure 33: Content of Outbound PDU details at the client.

From the content, the following can be seen:

- Source port: 1026
- Destination port: 80
- Sequence number: 108
- Acknowledgment number: 245
- Flags set: 1 (ACK)

To verify that this is the last segment to close the connection, capture then forward is clicked until the segment reaches the server. As seen by the content, there is no new segment generated (the sequence number, acknowledgement number, and the flags are the same as before.)

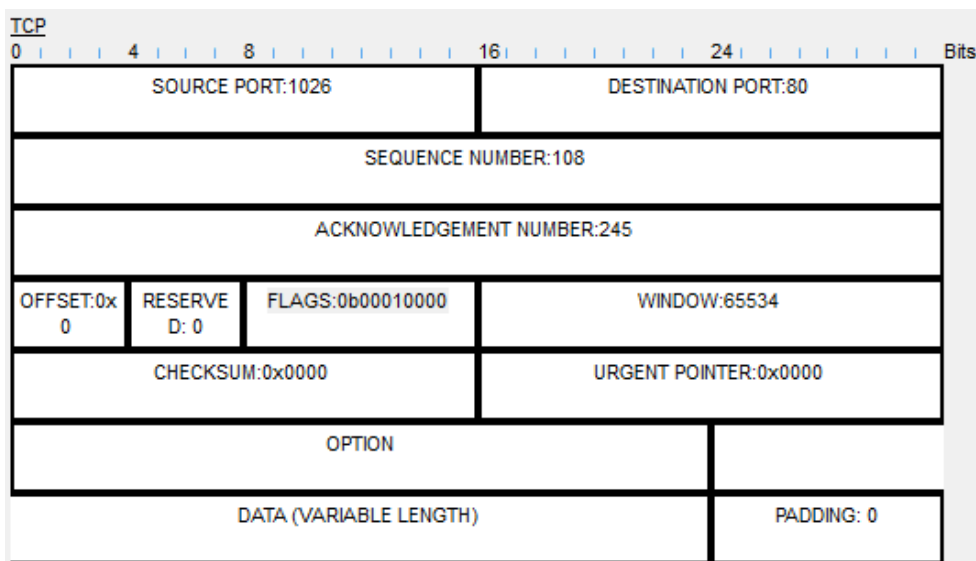


Figure 34: Content of Outbound PDU details at the Server indicating no new changes.

9. Final Questions

Question 1: What items does the UDP header contain?

The UDP header contains:

- **Source port:** identifies the sending application
- **Destination port:** identifies the receiving application
- **Length:** the length of the UDP header and data in bytes
- **Checksum:** used for error detection of the header and data

Question 2: What happens when the packet is lost during UDP communication in comparison with TCP?

- **UDP:** If a packet is lost, it is simply discarded, and no retransmission occurs.
- **TCP:** Detects lost packets using acknowledgments and retransmits them to ensure all data reaches the destination.

Question 3: What precedes and follows the data exchange via TCP protocol?

- **Precedes:** TCP connection establishment through the **three-way handshake**
- **Follows:** TCP connection termination through the **four-way handshake** or alternatively the abbreviated and faster **three-way handshake**.

Question 4: How is the TCP connection (virtual circuit) establishment called?

TCP connection establishment is called the **three-way handshake**, which involves SYN, SYN+ACK, and ACK messages.

Question 5: What groups of ports do the servers use for common services?

- **Well-known ports (0–1023):** These ports are used by servers providing common services. Clients commonly use them as destination ports when initiating communication.
- **Registered ports (1024–49151):** These ports are used by both clients and servers. For client purposes, some of this range may be reserved by the operating system. For server purposes, they must be registered by IANA.
- **Dynamic/Private ports (49152–65535):** These ports are dynamically assigned to clients when initiating communication.